NAME : Aditya pawar

Branch : AI&DS

Roll NO. : SYB 25

Exp No.1

AIM: Use sequence data types and their associated operations in Python a) Strings b) Lists c) Arrays d) Tuples e) Dictionary f) Sets g) Range

Theory :

1. What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types?

Ans: Sequence data types in Python are used to store multiple values in an ordered collection where each element has a specific position called an index. The main sequence data types are strings, lists, tuples, range and arrays. These data types allow accessing elements using indexing and extracting parts of data using slicing. Indexing means accessing elements using their position number starting from zero, while negative indexing allows accessing elements from the end of the sequence. Slicing is used to retrieve a subset of elements from a sequence using start, stop and step values. Indexing and slicing operations work similarly in strings, lists, tuples, arrays and range objects.

2. What is the difference between a list and a tuple in Python? Why would you prefer a tuple over a list in real-world applications?

ans: List = mutable ordered collection. Tuple = immutable ordered collection.

Lists and tuples are both used to store collections of elements in Python, but the main difference is that lists are mutable while tuples are immutable. This means elements inside a list can be modified after creation, whereas elements inside a tuple cannot be changed. Tuples are faster and consume less memory compared to lists. Tuples are preferred in real-world applications when data should remain fixed and protected from modification. They are commonly used for storing coordinates, configuration values and database records. Tuples can also be used as dictionary keys because of their immutability.

3)How are Python arrays different from lists? When should you use the array module instead of a list? Ans: Array = homogeneous collection List = heterogeneous collection.

Arrays and lists are both used to store multiple elements, but lists can store elements of different data types while arrays store elements of only the same data type. Arrays are more memory efficient and faster when handling large amounts of numeric data. Arrays require importing the array module, while lists are built into Python. Arrays are preferred when working with large numeric datasets, scientific calculations.

4. Explain how dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples?

Ans: Dictionary is an unordered collection of key-value pairs stored using hashing technique. Internally, dictionaries use a hash table mechanism in which a hash function converts the key into a unique memory location where the value is stored. This allows direct access to values using their keys. Dictionary lookups are faster compared to lists and tuples because dictionaries use hashing instead of sequential searching. As a result, dictionaries provide very efficient data retrieval and are widely used for fast searching and indexing operations.

5. What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios?

Ans: Sets and lists are both used to store collections of elements, but lists maintain order and allow duplicate values while sets do not maintain order and automatically remove duplicate values. Sets do not support indexing because they are unordered collections. Sets are useful when only unique elements are required and when performing mathematical operations such as union, intersection and difference. They are commonly used in removing duplicate data, membership testing, database filtering and data analysis tasks.

```
import array
# String Example
s = "Hello Python"
print("Length:", len(s))
print("Upper:", s.upper())
print("Lower:", s.lower())
print("Replace:", s.replace("Python", "World"))
print("Slice:", s[0:5])
#list operation
l = [10, 20, 30, 40]
l.append(50)
print("Append:", l)
l.insert(1, 15)
print("Insert:", l)
l.remove(30)
print("Remove:", l)
l.sort()
```

```
print("Sort:", l)
print("Length:", len(l))
#array operation
arr = array.array('i', [1, 2, 3, 4])
arr.append(5)
print("Append:", arr)
arr.insert(1, 10)
print("Insert:", arr)
arr.remove(3)
print("Remove:", arr)
arr.reverse()
print("Reverse:", arr)
print("Index of 10:", arr.index(10))
# Tuple Example
t = (1, 2, 3, 4, 2)
print("Length:", len(t))
print("Count of 2:", t.count(2))
print("Index of 3:", t.index(3))
print("Slice:", t[1:3])
t2 = t + (5, 6)
print("Concatenation:", t2)
# Dictionary Example
d = {"name": "Aditya", "age":19}
d["city"] = "Pune"
print("Add:", d)
d["age"] = 19
print("Update:", d)
d.pop("city")
print("Remove:", d)
print("Keys:", d.keys())
print("Values:", d.values())
# Set Example
s = {1, 2, 3, 4}
s.add(5)
print("Add:", s)
s.remove(2)
print("Remove:", s)
s2 = {4, 5, 6}
print("Union:", s.union(s2))
print("Intersection:", s.intersection(s2))
print("Length:", len(s))
# Range Example
r = range(1, 10)
print("Range:", list(r))
r2 = range(1, 10, 2)
print("Step:", list(r2))
print("Length:", len(r))
print("Element at index 3:", r[3])
print("List:", list(r))
```

```
Length: 12
Upper: HELLO PYTHON
Lower: hello python
Replace: Hello World
Slice: Hello
Append: [10, 20, 30, 40, 50]
Insert: [10, 15, 20, 30, 40, 50]
Remove: [10, 15, 20, 40, 50]
Sort: [10, 15, 20, 40, 50]
Length: 5
Append: array('i', [1, 2, 3, 4, 5])
Insert: array('i', [1, 10, 2, 3, 4, 5])
Remove: array('i', [1, 10, 2, 4, 5])
Reverse: array('i', [5, 4, 2, 10, 1])
Index of 10: 3
Length: 5
Count of 2: 2
Index of 3: 2
Slice: (2, 3)
Concatenation: (1, 2, 3, 4, 2, 5, 6)
Add: {'name': 'Aditya', 'age': 19, 'city': 'Pune'}
Update: {'name': 'Aditya', 'age': 19, 'city': 'Pune'}
Remove: {'name': 'Aditya', 'age': 19}
Keys: dict_keys(['name', 'age'])
Values: dict_values(['Aditya', 19])
Add: {1, 2, 3, 4, 5}
Remove: {1, 3, 4, 5}
Union: {1, 3, 4, 5, 6}
Intersection: {4, 5}
Length: 4
Range: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Step: [1, 3, 5, 7, 9]
Length: 9
Element at index 3: 4
List: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Conclusion: Sequence data types in Python are essential for storing and managing collections of data efficiently in an ordered structure. They support important operations like indexing and slicing, which make accessing and manipulating elements easy. Lists provide flexibility through modification, tuples ensure data safety through immutability, arrays offer efficient storage for numeric data, dictionaries enable fast data retrieval using key–value mapping, sets help maintain unique elements by removing duplicates, and range objects support efficient iteration. Together, these data structures improve program performance, organization, and reliability, making them very useful in real-world Python applications.